

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC



BÁO CÁO BÀI TẬP LỚN

**Tìm hiểu các phương pháp giải chính xác cho
bài toán định tuyến phương tiện có ràng
buộc khung thời gian (VRPTW)**

Sinh viên thực hiện	Mã sinh viên
Trần Đức Kiên (Leader)	23000131
Tạ Văn Nghĩa	23000145
Vũ Tiến Đạt	23000111

Hà Nội, 25 - 4 - 2026

Mục lục

1	GIỚI THIỆU CHUNG	1
1.1	Bối cảnh và đặt vấn đề	1
1.2	Mục tiêu	1
2	MÔ TẢ BÀI TOÁN	2
2.1	Bài toán VRP	2
2.2	Bài toán VRPTW	2
2.3	Input	3
2.4	Output	3
2.5	Các phương pháp giải chính xác cho bài toán VRPTW	3
2.6	Bộ dữ liệu của Solomon Benchmark	4
3	MÔ HÌNH HÓA BÀI TOÁN	6
3.1	Tập hợp và tham số	6
3.2	Biến quyết định	6
3.3	Hàm mục tiêu	6
3.4	Các ràng buộc	7
4	BRANCH AND BOUND	8
4.1	Khái quát	8
4.2	Ý tưởng thuật toán	8
4.3	Nguyên lý hoạt động	9
5	Branch-and-Cut	14
5.1	Khái quát	14
5.2	Ý tưởng thuật toán	14
5.3	Nguyên lý hoạt động	15
5.4	Pseudocode	16
5.5	Độ phức tạp	17
6	BRANCH AND PRICE	18
6.1	Khái quát	18

6.2	Ý tưởng thuật toán	18
6.3	Column Generation	19
6.3.1	Master Problem (RMP)	19
6.3.2	Subproblem: SPPRC	20
6.4	Shortest Path with Resource Constraints (SPPRC)	22
6.4.1	Khái quát	22
6.4.2	Nguyên lý hoạt động	22
6.5	Branch and Bound trong Branch and Price	25
6.5.1	Phân nhánh theo cạnh (Edge Branching)	25
6.5.2	Nguyên lý hoạt động	26
6.6	Độ phức tạp của thuật toán	29
7	KẾT QUẢ THỰC NGHIỆM	30
7.1	Thiết lập thực nghiệm	30
7.2	Kết quả so sánh	30
7.3	Phân tích kết quả	31
7.4	Nhận xét	31
8	TÀI LIỆU THAM KHẢO	32

Danh sách hình vẽ

Danh sách bảng

1	Kết quả so sánh các thuật toán với quy mô khách hàng nhỏ và trung bình . . .	30
---	--	----

1 GIỚI THIỆU CHUNG

1.1 Bối cảnh và đặt vấn đề

Trong bối cảnh ngành logistics và vận tải ngày càng phát triển, nhu cầu tối ưu hóa việc giao nhận hàng hóa trở nên đặc biệt quan trọng nhằm giảm chi phí vận hành, tiết kiệm thời gian và nâng cao chất lượng dịch vụ. Bài toán định tuyến phương tiện có ràng buộc khung thời gian (Vehicle Routing Problem with Time Windows – VRPTW) là một trong những bài toán tối ưu tổ hợp quan trọng trong lĩnh vực Operations Research và logistics.

VRPTW không chỉ yêu cầu xác định lộ trình tối ưu cho các phương tiện mà còn phải đảm bảo mỗi khách hàng được phục vụ trong khoảng thời gian quy định. Đây là bài toán có tính ứng dụng cao, độ phức tạp lớn thuộc lớp NP-hard. Vì vậy, việc tìm hiểu các phương pháp giải chính xác cho bài toán VRPTW có ý nghĩa quan trọng cả về mặt lý thuyết và ứng dụng thực tiễn.

Việc giải quyết bài toán VRPTW giúp:

- Giảm tổng quãng đường di chuyển
- Giảm chi phí nhiên liệu
- Tối ưu số lượng phương tiện sử dụng

1.2 Mục tiêu

Mục tiêu của bài báo cáo này là tìm hiểu tổng quan về bài toán định tuyến phương tiện có ràng buộc khung thời gian (Vehicle Routing Problem with Time Windows – VRPTW). Bên cạnh đó, xây dựng mô hình toán học cho bài toán VRPTW, bao gồm hàm mục tiêu, biến quyết định và các ràng buộc liên quan đến lộ trình và khung thời gian phục vụ.

Ngoài ra, báo cáo sẽ sử dụng một số phương pháp giải chính xác phổ biến được áp dụng cho bài toán VRPTW như Branch and Bound, Branch and Cut, Branch and Price và Branch and Cut and Price. Nhóm thực nghiệm trên bộ dữ liệu Solomon Benchmark thường được sử dụng để đánh giá hiệu quả của các thuật toán giải VRPTW. Thông qua đó, báo cáo phân tích và đánh giá ưu điểm, hạn chế của các phương pháp giải chính xác đối với bài toán này.

2 MÔ TẢ BÀI TOÁN

2.1 Bài toán VRP

Bài toán Định tuyến Phương tiện (Vehicle Routing Problem - VRP) là dạng tổng quát hóa của bài toán Người bán hàng (TSP), hướng tới việc xác định lộ trình di chuyển tối ưu cho một đội xe nhằm giảm thiểu chi phí hoặc thời gian giao hàng. Được Dantzig và Ramser giới thiệu lần đầu năm 1959 qua bài toán vận chuyển xăng dầu, VRP thuộc lớp tối ưu tổ hợp NP-khó, nghĩa là hiện chưa có thuật toán giải chính xác trong thời gian đa thức. Ngày nay, trước sự bùng nổ của thương mại điện tử, VRP ngày càng thu hút nhiều sự quan tâm nghiên cứu. Việc giải quyết hiệu quả bài toán này đóng vai trò cốt lõi giúp các doanh nghiệp vận tải tối đa hóa hiệu suất đội xe và cắt giảm tối đa chi phí vận hành.

Bài toán được xét trên một đồ thị có hướng $G = (V, A)$. Trong đó:

- $V = \{0, 1, \dots, n\}$ là tập hợp các đỉnh. Đỉnh 0 đại diện cho kho trung tâm (Depot), các đỉnh còn lại $N = \{1, \dots, n\}$ đại diện cho các khách hàng cần phục vụ.
- $A = \{(i, j) : i, j \in V, i \neq j\}$ là tập hợp các cung đường nối giữa các đỉnh, tương ứng với các lộ trình di chuyển khả thi của phương tiện.

2.2 Bài toán VRPTW

Bài toán Định tuyến Phương tiện có Ràng buộc Khung thời gian (Vehicle Routing Problem with Time Windows - VRPTW) là một biến thể mở rộng quan trọng của bài toán VRP. Ngoài mục tiêu tối ưu hóa lộ trình di chuyển của đội xe, bài toán còn yêu cầu mỗi khách hàng phải được phục vụ trong một khoảng thời gian xác định trước, gọi là khung thời gian (Time Window).

Bài toán được định nghĩa bởi một đội xe K , một tập khách hàng V và một đồ thị có hướng G . Đội xe được coi là đồng nhất (tất cả các xe đều giống hệt nhau). Đồ thị bao gồm $|C| + 1$ đỉnh, trong đó các khách hàng được đánh số là $1, 2, \dots, n$ và kho bãi (depot) đánh số là 0. Tập hợp tất cả các đỉnh $0, 1, \dots, n$ được ký hiệu là N .

$A = \{(i, j) : i, j \in V, i \neq j\}$ là tập hợp các tuyến đường khả thi giữa các đỉnh trong đồ thị. Với mỗi cung (i, j) , tồn tại một chi phí di chuyển c_{ij} và thời gian di chuyển tương ứng t_{ij} .

Mỗi xe có sức chứa là Q và mỗi khách hàng i có một nhu cầu hàng hóa q_i . Mỗi khách hàng i có một khung thời gian $[a_i, b_i]$, trong đó xe phải bắt đầu phục vụ khách hàng trước thời điểm b_i . Nếu xe đến trước thời điểm mở cửa a_i , xe phải chờ đến thời điểm a_i mới được phép bắt đầu phục vụ.

Khung thời gian của kho(depot) được ký hiệu là $[a_0, b_0]$, đại diện cho khoảng thời gian hoạt động của toàn bộ hệ thống giao hàng. Các phương tiện không được phép rời kho(depot) trước thời điểm a_0 và phải quay trở lại depot không muộn hơn thời điểm b_0 .

2.3 Input

Đầu vào của bài toán VRPTW gồm n địa điểm, bao gồm một kho chứa hàng (Depot) và tập $n - 1$ khách hàng. Ngoài ra, bài toán còn bao gồm:

2.4 Output

Output của bài toán VRPTW là một tập các tuyến đường cho đội xe sao cho mỗi tuyến đường bắt đầu và kết thúc tại depot, tất cả khách hàng đều được phục vụ đúng một lần và các ràng buộc về sức chứa phương tiện cũng như khung thời gian phục vụ đều được thỏa mãn. Mọi lộ trình đều bắt đầu tại đỉnh 0 và kết thúc tại đỉnh 0.

Mục tiêu của bài toán là tìm phương án tối ưu nhằm cực tiểu hóa tổng chi phí di chuyển của toàn bộ đội xe.

2.5 Các phương pháp giải chính xác cho bài toán VRPTW

Do bài toán VRPTW thuộc lớp bài toán NP-khó, việc tìm nghiệm tối ưu cho các bài toán có kích thước lớn là một thách thức đáng kể. Để giải quyết bài toán này, nhiều phương pháp giải chính xác đã được đề xuất nhằm đảm bảo tìm được nghiệm tối ưu toàn cục.

Các phương pháp giải chính xác cho bài toán VRPTW trong bài báo cáo này bao gồm:

- **Branch and Bound:** Phương pháp phân chia không gian nghiệm thành các bài toán con dưới dạng cây tìm kiếm, đồng thời sử dụng cận trên và cận dưới để loại bỏ các nhánh không tiềm năng.
- **Branch and Cut:** Là sự kết hợp giữa Branch and Bound và các kỹ thuật Cutting Planes nhằm loại bỏ các nghiệm không hợp lệ và tăng tốc quá trình hội tụ.

- **Branch and Price:** Kết hợp giữa Branch and Bound và kỹ thuật Column Generation, thường được sử dụng cho các mô hình có số lượng biến rất lớn như bài toán VRPTW.
- **Branch and Cut and Price:** Là phương pháp kết hợp đồng thời Branch and Bound, Cutting Planes và Column Generation nhằm tận dụng ưu điểm của cả hai kỹ thuật Branch and Cut và Branch and Price.

2.6 Bộ dữ liệu của Solomon Benchmark

Trong báo cáo này, nhóm sử dụng bộ dữ liệu Solomon benchmark để thực nghiệm và đánh giá các phương pháp giải chính xác bài toán VRPTW. Đây là bộ dữ liệu chuẩn được đề xuất bởi Solomon vào năm 1987 và được sử dụng rộng rãi trong nhiều nghiên cứu về bài toán định tuyến phương tiện có ràng buộc khung thời gian.

Bộ dữ liệu Solomon gồm nhiều bộ test khác nhau, mỗi bộ bao gồm:

- Một depot trung tâm.
- Tập các khách hàng với tọa độ vị trí trong mặt phẳng.
- Nhu cầu hàng hóa của từng khách hàng.
- Khung thời gian phục vụ (time window).
- Thời gian phục vụ tại mỗi khách hàng.
- Số lượng phương tiện và sức chứa tối đa của mỗi xe.

Các bộ dữ liệu trong Solomon Benchmark được chia thành ba nhóm chính:

- **C (Clustered):** Các khách hàng được phân bố thành từng cụm.
- **R (Random):** Các khách hàng được phân bố ngẫu nhiên.
- **RC (Random-Clustered):** Kết hợp giữa phân bố cụm và ngẫu nhiên.

Trong đó, báo cáo sử dụng bộ dữ liệu thuộc nhóm C (Clustered). Bộ dữ liệu này gồm một depot và 10 khách hàng được phân bố thành các cụm gần nhau, giúp đánh giá hiệu quả của các thuật toán tối ưu trong môi trường có cấu trúc dữ liệu cụ thể.

Mỗi dòng dữ liệu của khách hàng trong bộ dữ liệu bao gồm các thông tin:

- Mã khách hàng.
- Tọa độ (x,y) .
- Nhu cầu hàng hóa.
- Thời gian mở cửa và đóng cửa.
- Thời gian phục vụ.

Bộ dữ liệu Solomon đã được sử dụng và xem như tiêu chuẩn để kiểm tra về hiệu suất, chất lượng của thuật toán giải quyết bài toán VRP. Các nhà nghiên cứu thường dùng bộ dữ liệu trên cũng như các phiên bản mở rộng của nó để so sánh, đánh giá các thuật toán khác nhau.

3 MÔ HÌNH HÓA BÀI TOÁN

Để giải quyết bài toán, chúng tôi xây dựng mô hình toán học như sau:

3.1 Tập hợp và tham số

- $V = \{0, 1, 2, \dots, n\}$: 0 là Depot (điểm bắt đầu và kết thúc của tất cả các xe). $1, 2, \dots, n$ là tập các khách hàng.
- $K = \{1, 2, \dots, m\}$: Tập hợp tất cả các xe.
- q_i : Nhu cầu của mỗi điểm i ($q_0 = 0$). Q : Sức chứa của xe.
- c_{ij}, t_{ij} : Chi phí (khoảng cách) và thời gian di chuyển từ i đến j .
- s_i : Thời gian phục vụ tại điểm i .
- $[a_i, b_i]$: Khung thời gian phục vụ tại điểm i .

Lưu ý: ta sẽ giả sử tốc độ di chuyển của xe mặc định là 1, nghĩa là với $t_{ij} = 1$ đơn vị thời gian thì xe sẽ di chuyển được $c_{ij} = 1$ đơn vị khoảng cách. Do đó, $c_{ij} = t_{ij} \quad \forall i, j$.

3.2 Biến quyết định

- $x_{ij} \in \{0, 1\}$: Bằng 1 nếu có một xe bất kỳ di chuyển trực tiếp từ đỉnh i đến đỉnh j , bằng 0 nếu ngược lại.
- $s_i \geq 0$: Biến liên tục thể hiện thời gian bắt đầu phục vụ tại đỉnh i .

3.3 Hàm mục tiêu

Mục tiêu của bài toán là cực tiểu hóa tổng chi phí (hoặc tổng quãng đường) di chuyển của toàn bộ đội xe:

$$\text{Minimize } Z = \sum_{i \in V} \sum_{j \in V, j \neq i} c_{ij} x_{ij} \quad (1)$$

3.4 Các ràng buộc

- Mỗi khách hàng được phục vụ đúng một lần.

$$\sum_{j \in V, j \neq i} x_{ij} = 1 \quad \forall i \in V \setminus \{0\} \quad (2)$$

- Đảm bảo bảo toàn luồng (xe đến điểm i thì phải rời khỏi điểm i).

$$\sum_{i \in V, i \neq h} x_{ih} = \sum_{j \in V, j \neq h} x_{hj} \quad \forall h \in V \setminus \{0\} \quad (3)$$

- Đảm bảo số lượng xe tối đa là m (xe).

$$\sum_{j \in V \setminus \{0\}} x_{0j} \leq m \quad (4)$$

- Ràng buộc thời gian. Đảm bảo thời gian bắt đầu phục vụ s_i nằm trong khoảng $[a_i, b_i]$ và đảm bảo thứ tự thời gian hợp lệ. M là số rất lớn (Big-M). $M_{ij} = b_i + t_{ij} - a_j$

$$a_i \leq s_i \leq b_i \quad \forall i \in V \quad (5)$$

$$s_i + t_{ij} - s_j + s_i^{service} \leq M(1 - x_{ij}), \quad \forall i, j \in V \quad (6)$$

- Ràng buộc sức chứa. Một phương tiện chỉ có thể chứa được tối đa bằng sức tải của xe.

$$u_i + q_j - u_j \leq Q(1 - x_{ij}), \quad \forall i, j \in V \quad (7)$$

$$q_i \leq u_i \leq Q, \quad \forall i \in V \setminus \{0\} \quad (8)$$

- Ràng buộc nguyên

$$x_{ij} \in \{0, 1\}, \quad \forall i, j \in V \quad (9)$$

4 BRANCH AND BOUND

4.1 Khái quát

Branch and Bound (Nhánh và Cận) là một phương pháp giải chính xác phổ biến cho các bài toán tối ưu tổ hợp và quy hoạch nguyên. Ý tưởng chính của Branch and Bound là phân chia bài toán ban đầu thành nhiều bài toán con nhỏ hơn dưới dạng cây tìm kiếm (*Branching*). Với mỗi bài toán con, thuật toán sẽ tính toán các cận trên hoặc cận dưới (*Bounding*) nhằm đánh giá khả năng chứa nghiệm tối ưu của nhánh đó. Nếu một nhánh được xác định chắc chắn không thể tạo ra nghiệm tốt hơn nghiệm hiện tại, nhánh đó sẽ bị loại bỏ khỏi quá trình tìm kiếm (*Pruning*).

Phương pháp Branch and Bound thường bắt đầu bằng việc giải bài toán thư giãn tuyến tính (LP Relaxation). Nếu nghiệm thu được là nghiệm nguyên thì đây chính là nghiệm tối ưu của bài toán. Ngược lại, nếu nghiệm chưa nguyên, thuật toán sẽ lựa chọn một biến phân số để thực hiện phân nhánh, từ đó tạo ra các bài toán con mới với các ràng buộc bổ sung.

4.2 Ý tưởng thuật toán

Thuật toán Branch and Bound hoạt động dựa trên ý tưởng phân chia không gian nghiệm của bài toán thành nhiều bài toán con nhỏ hơn dưới dạng cây tìm kiếm. Mỗi nút trên cây đại diện cho một tập nghiệm khả thi của bài toán.

Đối với bài toán VRPTW, nếu thực hiện vét cạn toàn bộ các phương án phân công xe và lộ trình di chuyển thì số lượng nghiệm sẽ tăng rất nhanh theo cấp số mũ khi số khách hàng tăng lên. Điều này khiến việc tìm kiếm trực tiếp nghiệm tối ưu trở nên không khả thi đối với các bộ dữ liệu thực tế. Vì vậy, Branch and Bound được sử dụng nhằm giảm không gian tìm kiếm bằng cách loại bỏ sớm các nhánh không tiềm năng.

Ý tưởng của thuật toán có thể được mô tả qua các bước sau:

1. Phân nhánh (Branching)

Bài toán VRPTW được mô hình hóa dưới dạng bài toán quy hoạch nguyên hỗn hợp (MILP) với các biến nhị phân $x_{ij} \in \{0, 1\}$. Khi giải LP Relaxation (nới lỏng điều kiện nguyên thành $0 \leq x_{ij} \leq 1$), nếu xuất hiện biến phân số, ví dụ $x_{12} = 0.43$, thuật toán sẽ tạo ra hai bài toán con:

- Nhánh thứ nhất: thêm ràng buộc $x_{12} = 0$
- Nhánh thứ hai: thêm ràng buộc $x_{12} = 1$

Quá trình này tiếp tục đệ quy cho đến khi thu được nghiệm nguyên hoặc nhánh bị loại bỏ.

2. Tính cận (Bounding)

Tại mỗi nút, thuật toán giải LP Relaxation để tính cận dưới (Lower Bound - LB) cho bài toán con tương ứng. Giá trị LB này cho biết nghiệm tốt nhất có thể đạt được trong vùng không gian nghiệm của nhánh đó, từ đó làm cơ sở để quyết định có tiếp tục khám phá nhánh hay không.

3. Cắt tỉa (Pruning)

Một nhánh sẽ bị loại bỏ sớm mà không cần khám phá thêm nếu:

- Bài toán con vô nghiệm (vi phạm ràng buộc sức chứa hoặc time window).
- Cận dưới của nhánh lớn hơn hoặc bằng nghiệm tốt nhất hiện tại, tức là nhánh này chắc chắn không thể cho nghiệm tối ưu hơn.
- Nghiệm LP đã là nghiệm nguyên, không cần phân nhánh thêm.

Quá trình trên lặp lại cho đến khi toàn bộ cây tìm kiếm được duyệt hết. Nghiệm nguyên tốt nhất tìm được chính là nghiệm tối ưu của bài toán VRPTW.

4.3 Nguyên lý hoạt động

Thuật toán Branch and Bound hoạt động thông qua một cây tìm kiếm, trong đó mỗi nút đại diện cho một bài toán nối lỏng với các ràng buộc được cập nhật liên tục. Quy trình hoạt động gồm các bước sau:

1. Khởi tạo (Initialization)

Khởi tạo cây B&B với một nút gốc duy nhất chứa toàn bộ mô hình toán học ban đầu với điều kiện biến x_{ij} đã được nối lỏng. Khởi tạo giá trị cận trên $UB = +\infty$, đóng vai trò lưu trữ giá trị hàm mục tiêu của nghiệm nguyên tốt nhất tìm được trong quá trình duyệt.

2. Lựa chọn nút (Node Selection)

Kiểm tra tập hợp các nút đang chờ khám phá. Nếu tập hợp này rỗng, thuật toán kết thúc. Ngược lại, trích xuất một nút từ hàng đợi để tiến hành xử lý.

3. Đánh giá cận dưới (Bounding)

Tại nút đang xét, tiến hành giải bài toán LP Relaxation. Gọi giá trị hàm mục tiêu thu được là Z_{LP} , đây chính là cận dưới (Lower Bound) của nút hiện tại.

4. Cắt tỉa (Pruning)

Nút hiện tại sẽ bị loại bỏ nếu rơi vào một trong ba trường hợp:

- **Prune by Infeasibility:** Bài toán LP vô nghiệm do vi phạm ràng buộc sức chứa hoặc time window.
- **Prune by Bound:** Cận dưới $Z_{LP} \geq UB$, chứng tỏ vùng này chỉ chứa các phương án kém tối ưu hơn nghiệm hiện tại.
- **Prune by Optimality:** Nghiệm LP trả về là nghiệm nguyên (tất cả $x_{ij} \in \{0, 1\}$). Nếu $Z_{LP} < UB$, cập nhật $UB = Z_{LP}$ và lưu lại lịch trình tương ứng.

5. Phân nhánh (Branching)

Nếu nút không bị cắt tỉa, nghiệm LP chứa biến phân số $x_{ij} \notin \{0, 1\}$. Thuật toán chọn một biến phân số để phân nhánh, tạo ra hai bài toán con:

- **Nhánh trái:** Thêm ràng buộc $x_{ij} = 0$, cấm xe di chuyển trực tiếp từ i đến j .
- **Nhánh phải:** Thêm ràng buộc $x_{ij} = 1$, bắt buộc xe di chuyển trực tiếp từ i đến j .

Hai nút con được đẩy vào hàng đợi và thuật toán quay lại Bước 2.

6. Kết thúc (Termination)

Thuật toán dừng khi toàn bộ cây tìm kiếm đã được duyệt hết. Nghiệm nguyên ứng với giá trị UB cuối cùng chính là phương án tối ưu cho bài toán VRPTW.

Pseudocode

Input: Đồ thị $G = (V, A)$, khách hàng N , xe K , sức chứa Q ,
time windows $[a_i, b_i]$

Output: Lộ trình tối ưu với tổng chi phí nhỏ nhất

BranchAndBound():

 best_val = $+\infty$

 best_sol = NULL

 root = LP_Relaxation(bài toán gốc)

 queue.push(root)

 while queue không rỗng:

 node = queue.pop() // Best-First hoặc Depth-
 First

 // Bounding

 Z_LP = SolveLP(node)

 // Pruning

 if Z_LP == vô nghiệm:

 continue // Prune

 if Z_LP $\geq$ best_val:

 continue // Prune

 if tất cả $x_{ij} \in \{0, 1\}$:

 best_val = Z_LP // Prune by Optimality

 best_sol = node.solution

 continue

 // Branching: chọn biến phân số (ví dụ $x_{ij} = 0.43$)


```

    xij = chon_bien_phan_so(node)

    left = node.copy()
    left.add_constraint(xij = 0) // Nhanh trai: cam canh
        (i,j)
    queue.push(left)

    right = node.copy()
    right.add_constraint(xij = 1) // Nhanh phai: bat buoc
        canh (i,j)
    queue.push(right)

    return best_val, best_sol

```

SolveLP(node):

```

    // Giai LP Relaxation: 0 <= xij <= 1

```

Minimize $Z = \sum_i \sum_{j \neq i} c_{ij} x_{ij}$

Subject to:

$\sum_j x_{ij} = 1 \quad \forall i \in N$ // moi KH phuc vu dung 1 lan

$\sum_j x_{hj} = \sum_i x_{ih} \quad \forall h \in N$ // bao toan luong

$\sum_j x_{0j} \leq m$ // so xe toi da

$a_i \leq s_i \leq b_i \quad \forall i \in V$ // time window

$s_i + t_{ij} - s_j \leq M(1 - x_{ij}) \quad \forall i, j$ // thu tu thoi gian

$u_i + q_j - u_j \leq Q(1 - x_{ij}) \quad \forall i, j$ // suc chua

$0 \leq x_{ij} \leq 1 \quad \forall i, j \in V$ // noi long nguyen

+ cac rang buoc bo sung cua node

```

    return Z, solution

```

Độ phức tạp của thuật toán:

- Gọi $n = |V|$ là số đỉnh trong đồ thị (bao gồm depot và các khách hàng).
- **Số biến:** Mô hình VRPTW có n^2 biến nhị phân x_{ij} , do đó trong trường hợp xấu nhất, cây tìm kiếm B&B có thể có tối đa 2^{n^2} nút.
- **Chi phí mỗi nút:** Tại mỗi nút, thuật toán giải một bài toán LP Relaxation. Sử dụng thuật toán Simplex, chi phí giải một LP có m ràng buộc và n^2 biến là $O((m \cdot n^2)^{1.5})$.
- **Độ phức tạp tổng thể:**

$$T(n) = O\left(2^{n^2} \cdot (m \cdot n^2)^{1.5}\right)$$

Tuy nhiên, nhờ các kỹ thuật cắt tỉa, số nút thực tế được duyệt thường nhỏ hơn rất nhiều so với trường hợp lý thuyết.

5 Branch-and-Cut

5.1 Khái quát

Branch-and-Cut là một phương pháp nâng cao của Branch-and-Bound, được sử dụng rộng rãi để giải các bài toán tối ưu tổ hợp khó như VRPTW. Phương pháp này kết hợp hai kỹ thuật : **Branch-and-Bound** (phân nhánh và cận) và **Cutting Planes** (thêm các mặt cắt).

Ý tưởng cốt lõi của Branch-and-Cut là không chỉ phân nhánh khi gặp nghiệm phân số mà còn chủ động tìm và thêm các ràng buộc cắt (valid inequalities) vào mô hình LP relaxation tại từng nút của cây tìm kiếm. Những ràng buộc cắt này giúp thắt chặt vùng khả thi, nâng cao chất lượng cận dưới (lower bound) một cách đáng kể, từ đó tăng hiệu quả cắt tỉa và giảm mạnh số lượng nút cần khám phá so với Branch-and-Bound thuần túy.

Trong đề tài này, Branch-and-Cut được triển khai tùy chỉnh trên nền tảng Google OR-Tools (GLOP solver) vì solver này không hỗ trợ MIP tự động. Thuật toán tập trung vào việc tách các cắt dựa trên vi phạm subtour và ràng buộc dung lượng (capacity cuts).

5.2 Ý tưởng thuật toán

Branch-and-Cut hoạt động dựa trên bốn thành phần chính:

1. **Phân nhánh (Branching)**: Khi nghiệm LP relaxation chứa biến phân số $x_{ij}^* \notin \{0, 1\}$, thuật toán tạo hai nhánh con bằng cách thêm ràng buộc $x_{ij} = 0$ hoặc $x_{ij} = 1$.
2. **Tạo mặt cắt (Cutting Planes)**: Sau khi giải LP, thuật toán thực hiện separation để phát hiện và thêm các ràng buộc cắt hợp lệ (capacity cuts / subtour elimination constraints) nhằm thắt chặt vùng nói lỏng.
3. **Tính cận (Bounding)**: Tại mỗi nút, giải LP relaxation để tính cận dưới Z_{LP} . Cận dưới này càng lớn (nhờ các cut được thêm) thì khả năng cắt tỉa càng cao.
4. **Cắt tỉa (Pruning)**: Nhánh bị loại bỏ nếu rơi vào một trong các trường hợp: vô nghiệm, $Z_{LP} \geq UB$, hoặc nghiệm đã nguyên (integral).

5.3 Nguyên lý hoạt động

Quy trình Branch-and-Cut trong triển khai được thực hiện đệ quy theo chiều sâu (Depth-First Search) với các bước sau tại mỗi nút:

1. Giải bài toán LP relaxation hiện tại để thu được nghiệm phân số x^* và giá trị cận dưới Z_{LP} .
2. Thực hiện vòng lặp Cutting Plane:
 - Sử dụng Union-Find để xây dựng các thành phần liên thông S trong support graph (với ngưỡng $\varepsilon = 10^{-6}$).
 - Với mỗi tập S ($|S| \geq 2$), tính:

$$d(S) = \sum_{i \in S} d_i, \quad k(S) = \left\lceil \frac{d(S)}{Q} \right\rceil$$

$$\text{flowOut}(S) = \sum_{i \in S} \sum_{j \notin S} x_{ij}^*$$

- Nếu $\text{flowOut}(S) < k(S)$, nghĩa là tổng luồng ra khỏi tập S chưa đủ để phục vụ nhu cầu của S , ta thêm ràng buộc cắt:

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq k(S)$$

Ví dụ: Giả sử một tập S có tổng nhu cầu $d(S) = 45$, dung lượng xe $Q = 20$. Khi đó $k(S) = \lceil 45/20 \rceil = 3$. Nghĩa là phải có ít nhất 3 xe rời khỏi tập S .

3. Cập nhật gap = $\frac{UB - Z_{LP}}{UB}$. Gap thể hiện khoảng cách tương đối giữa nghiệm tốt nhất hiện tại (upper bound) và cận dưới tại nút đang xét. Thuật toán chỉ tiếp tục thực hiện cutting plane khi gap $> 5\%$ nhằm cân bằng giữa chất lượng lower bound và thời gian tính toán.
4. Nếu nghiệm LP là nguyên (tất cả $x_{ij} \in \{0, 1\}$), cập nhật upper bound UB và lưu nghiệm tốt nhất nếu $Z_{LP} < UB$.
5. Nếu chưa nguyên, chọn biến x_{ij} gần 1 nhất để phân nhánh, giải thử LP trên hai nhánh trước khi đệ quy.

5.4 Pseudocode

```

BranchAndCut():
    UB = GreedyNearestNeighbor()           // Upper Bound ban đầu
    best_sol = NULL

    root = LP_Relaxation(bai toan goc)
    manualBranchAndCut(root, depth = 0)

manualBranchAndCut(node):
    Solve LP relaxation → Z_LP

    // Cutting Plane Phase
    while gap > 0.05:
        S = findViolatedSubtour(x*, instance)
        if S == null: break

        k_S = ceil( sum d_i / Q for i in S )
        Add cut: sum x_ij (i in S, j not in S) >= k_S
        Re-solve LP
        Update Z_LP and gap

    if infeasible or Z_LP >= UB - epsilon: return

    if solution is integer:
        if Z_LP < UB:
            UB = Z_LP
            best_sol = current solution
        return

    (i,j) = select_branch_variable_closest_to_one()
    Branch on x_ij = 1 and x_ij = 0

```

5.5 Độ phức tạp

Về mặt lý thuyết, Branch-and-Cut có độ phức tạp tương tự Branch-and-Bound:

$$T(n) = O\left(2^{n^2} \cdot T_{LP}(m, n^2)\right)$$

trong đó T_{LP} là thời gian giải một bài toán LP.

Tuy nhiên, nhờ cơ chế thêm cutting planes động và separation heuristic hiệu quả dựa trên Union-Find + HashSet, số lượng nút thực tế được duyệt giảm đáng kể.

6 BRANCH AND PRICE

6.1 Khái quát

Branch and Price (Nhánh và Giá) là một phương pháp giải chính xác mạnh mẽ cho các bài toán quy hoạch nguyên, kết hợp hai kỹ thuật cốt lõi: *Branch and Bound* (Nhánh và Cận) và *Column Generation* (Sinh cột). Thay vì giải trực tiếp bài toán với toàn bộ các biến, Branch and Price chỉ làm việc với một tập con các biến (cột) tại mỗi bước, và liên tục bổ sung các biến có tiềm năng cải thiện nghiệm thông qua bài toán con (Subproblem).

Đối với bài toán VRPTW, mỗi biến trong mô hình Master Problem tương ứng với một tuyến đường khả thi (route) thỏa mãn ràng buộc sức chứa và cửa sổ thời gian. Vì số lượng tuyến đường khả thi có thể rất lớn (tăng theo cấp số mũ với số khách hàng), Column Generation được dùng để chỉ sinh ra các tuyến đường thực sự có ích, trong khi Branch and Bound đảm bảo nghiệm cuối cùng là nguyên.

6.2 Ý tưởng thuật toán

Thuật toán Branch and Price hoạt động dựa trên cấu trúc cây tìm kiếm. Tại mỗi nút của cây, thay vì giải một LP Relaxation đơn giản, thuật toán giải một *Restricted Master Problem* (RMP) thông qua Column Generation. Quy trình tổng quát gồm:

1. Khởi tạo (Initialization)

Tại nút gốc, khởi tạo RMP với một tập các tuyến đường ban đầu (initial routes). Trong cài đặt này, mỗi khách hàng i được phục vụ bởi một tuyến đường đơn $[0, i, N-1]$ (từ depot đến khách hàng i rồi về depot), đảm bảo bài toán có nghiệm khởi tạo khả thi:

```
for i in range(user_param.nbclients - 2):
    route_cost = dist[0][i+1] + dist[i+1][N-1]
    route = Route(path=[0, i+1, N-1], cost=route_cost, Q
                  =1.0)
    init_routes.append(route)
```

2. Column Generation tại mỗi nút

Tại mỗi nút của cây B&B, thuật toán gọi `ColumnGeneration.compute_col_gen()` để giải RMP và sinh thêm các cột (tuyến đường) mới có reduced cost âm. Quá trình này lặp lại cho đến khi không còn cột nào có thể cải thiện nghiệm.

3. Phân nhánh theo cạnh (Edge Branching)

Nếu nghiệm LP của RMP chứa biến phân số, thuật toán phân nhánh theo một cạnh (i, j) trong đồ thị:

- **Nhánh 0:** Cấm cạnh (i, j) , đặt $\text{dist}[i][j] = +\infty$.
- **Nhánh 1:** Bắt buộc cạnh (i, j) , cấm tất cả cạnh ra khỏi i và vào j ngoại trừ cạnh (i, j) .

4. Cắt tỉa (Pruning)

Một nút bị loại bỏ nếu: bài toán con vô nghiệm, cận dưới tại nút vượt quá cận trên toàn cục, hoặc nghiệm đã nguyên (cập nhật cận trên nếu tốt hơn).

6.3 Column Generation

6.3.1 Master Problem (RMP)

Master Problem là một bài toán Set Covering/Partitioning với số cột tăng dần. Mô hình toán học:

$$\begin{aligned} \min \quad & \sum_{r \in R} c_r y_r \\ \text{s.t.} \quad & \sum_{r \in R} a_{ir} y_r \geq 1 \quad \forall i \in N \setminus \{0, N-1\} \\ & y_r \geq 0 \quad \forall r \in R \end{aligned}$$

Trong đó:

- R : tập các tuyến đường (cột) hiện có trong RMP.
- c_r : chi phí (tổng khoảng cách) của tuyến đường r .
- $a_{ir} = 1$ nếu khách hàng i nằm trong tuyến đường r , ngược lại bằng 0.
- $y_r \geq 0$: tỉ lệ sử dụng tuyến đường r (được nói lỏng từ nhị phân).

Bài toán được giải bằng Gurobi thông qua mô hình liên tục (CONTINUOUS), với các cột mới được thêm tăng dần (incremental column addition) thay vì tái tạo lại toàn bộ mô hình mỗi vòng lặp:

```
col = gp.Column()
for client_idx, constr in constraints.items():
    if client_idx in new_route.path[1:-1]:
        col.addTerms(1.0, constr)
y[idx] = model.addVar(vtype=GRB.CONTINUOUS, lb=0.0,
                      obj=cost, column=col, name=f"y_{idx}")
```

6.3.2 Subproblem: SPPRC

Bài toán con (Subproblem) trong Column Generation là *Shortest Path Problem with Resource Constraints* (SPPRC) – tìm tuyến đường có reduced cost âm nhỏ nhất. Reduced cost của một tuyến đường r được tính:

$$\bar{c}_r = c_r - \sum_{i \in N \setminus \{0, N-1\}} \pi_i \cdot a_{ir}$$

Trong đó π_i là giá trị đối ngẫu (dual variable) của ràng buộc khách hàng i trong RMP. Ma trận chi phí được cập nhật trước mỗi lần giải SPPRC:

$$\text{cost}[i][j] = \text{dist}[i][j] - \pi_i \quad \forall i \in N \setminus \{0, N-1\}$$

Pseudocode Column Generation:

```
Input:  RMP voi tap route ban dau R_0, ma tran dist, ma tran
        ttime
Output: Giai phap LP toi uu va gia tri ham muc tieu

ColumnGeneration(initial_routes):
    Tao RMP voi cac route trong initial_routes

    while True:
```

```
Giai RMP (Gurobi) => lay gia tri ham muc tieu Z_LP

// Lay dual variables  $\pi_i$  tu RMP
 $\pi_i$  = RMP.constraints[i].Pi voi moi i

// Cap nhat reduced cost cho SPPRC
for i in khach hang:
    for j in tat ca node:
        cost[i][j] = dist[i][j] -  $\pi_i$ 

// Giai SPPRC tim route co reduced cost am
new_routes = SPPRC.shortestPath(cost, dist, ttime)

if new_routes rong:
    break // Toi uu, khong co cot moi

// Them cac route moi vao RMP (incremental)
for route in new_routes:
    tinh cost_r = sum(dist[path[k]][path[k+1]])
    tao Column col tu cac rang buoc khach hang
    them bien y moi vao RMP voi col va obj=cost_r

// Cap nhat gia tri Q cho moi route
for route in routes:
    route.Q = y[route].X

return Z_LP, routes
```

6.4 Shortest Path with Resource Constraints (SPPRC)

6.4.1 Khái quát

SPPRC là bài toán tìm đường đi ngắn nhất từ depot 0 đến depot cuối $N - 1$ trên đồ thị có hướng, thỏa mãn đồng thời ba ràng buộc tài nguyên:

- **Reduced cost:** tổng chi phí tuyến đường (mục tiêu tối thiểu hóa).
- **Cửa sổ thời gian (Time Window):** thời gian đến khách hàng i phải nằm trong $[a_i, b_i]$.
- **Sức chứa (Capacity):** tổng nhu cầu trên tuyến đường $\leq Q$.

Thuật toán sử dụng kỹ thuật *Label Setting* (Irnich & Desaulniers), trong đó mỗi *label* L đại diện cho một đường đi khả thi từ depot đến nút hiện tại, lưu trữ trạng thái đầy đủ:

$$L = (v, \text{cost}, \text{ttime}, \text{demand}, \text{visited})$$

Với:

- v : nút hiện tại.
- cost : tổng reduced cost tích lũy.
- ttime : thời gian đến nút v (bao gồm thời gian chờ và phục vụ).
- demand : tổng nhu cầu tích lũy.
- $\text{visited}[i]$: mảng boolean cho biết khách hàng i đã được thăm chưa.

6.4.2 Nguyên lý hoạt động

Khởi tạo: Tạo label gốc tại depot 0 với $\text{cost} = 0$, $\text{ttime} = 0$, $\text{demand} = 0$.

Mở rộng label (REF – Resource Extension Function): Với label L tại nút u , mở rộng sang nút v nếu chưa thăm và cạnh (u, v) khả dụng ($\text{dist}[u][v] < \text{verybig}$):

$$\text{ttime}' = \max(a_v, \text{ttime} + \text{ttime}[u][v] + s_u)$$

$$\text{demand}' = \text{demand} + d_v$$

$$\text{cost}' = \text{cost} + \text{cost}[u][v]$$

Label mới được chấp nhận nếu $ttime' \leq b_v$ và $demand' \leq Q$.

Kỹ thuật tăng tốc (Feillet 2004): Sau khi mở rộng đến nút v , đánh dấu trước các khách hàng j không thể thăm tiếp theo (vì vi phạm time window hoặc capacity), giảm số label cần xét:

```
for j in range(1, nbclients - 1):
    if not newcust[j]:
        tt2 = tt + ttime[i][j] + s[i]
        d2 = d + d[j]
        if tt2 > b[j] or d2 > capacity:
            newcust[j] = True # đánh dấu là đã tham (khoa
                               lai)
```

Kiểm tra dominance: Label L_1 tại nút v *dominate* label L_2 cùng nút nếu:

$$L_1.cost \leq L_2.cost \wedge L_1.ttime \leq L_2.ttime \wedge L_1.demand \leq L_2.demand$$

$$\wedge \forall k : L_1.visited[k] \Rightarrow L_2.visited[k]$$

Label bị dominate sẽ bị loại bỏ khỏi tập U (unprocessed labels), giảm đáng kể không gian tìm kiếm.

Thu thập nghiệm: Các label có $cost < 0$ đến depot cuối được đưa vào tập P (processed solutions). Sau khi hoàn tất, truy vết ngược theo `index_prev_label` để tái tạo tuyến đường:

```
route.add_city(labels[lab].city)
path = labels[lab].index_prev_label
while path >= 0:
    route.add_city(labels[path].city)
    path = labels[path].index_prev_label
route.switch_path() # đảo ngược để có thu tu dung
```

Pseudocode SPPRC:

Input: Đồ thị $G=(V,A)$, `cost[][]`, `dist[][]`, `ttime[][]`, `a[]`,
`b[]`, `d[]`, `s[]`, Q

Output: Danh sách các route có reduced cost âm

SPPRC():

```

label_0 = (city=0, cost=0, ttime=0, demand=0, visited
           ={0:True})
U = SortedSet([label_0])    // tap cac label chua xu ly
P = SortedSet()             // tap cac label du ieu kien
                              (den depot)
city2labels[0] = [label_0]

while U khong rong va nbsol < maxsol:
    L = U.pop_first()       // lay label co cost nho nhat

    // Kiem tra dominance voi cac label cung nhan
    for L' in city2labels[L.city]:
        if L dominates L': danh dau L'.dominated=True,
            xoa khoi U
        if L' dominates L: danh dau L.dominated=True,
            break

    if L.dominated: tiep tuc

    if L.city == N-1:       // den depot cuoi
        if L.cost < 0: them L vao P, cap nhat nbsol
        tiep tuc

    // Mo rong label sang cac nhan lang gieng
    for v in V:
        if not L.visited[v] and dist[L.city][v] <
            verybig:
            tt' = max(a[v], L.ttime + ttime[L.city][v] +
                      s[L.city])
            d'  = L.demand + d[v]
            if tt' <= b[v] and d' <= Q:
                // Ap dung Feillet 2004: khoa truoc cac

```

```

        nhan khong the tham
        visited' = update_visited(tt', d', v)
        L' = new label(v, cost+cost[L.city][v],
            tt', d', visited')
        them L' vao U va city2labels[v]

// Truy vet duong di tu P
for lab in P:
    if not lab.dominated and lab.cost < -1e-4:
        tao route bang cach truy vet index_prev_label
        them route vao danh sach ket qua

return routes

```

6.5 Branch and Bound trong Branch and Price

6.5.1 Phân nhánh theo cạnh (Edge Branching)

Sau khi Column Generation hội tụ mà nghiệm còn phân số, cần phân nhánh. Biến phân số được chọn là cạnh (i^*, j^*) có giá trị x_{ij} xa 0 và 1 nhất, ưu tiên theo tiêu chí:

$$(i^*, j^*) = \arg \max_{(i,j): 0 < x_{ij} < 1} \min(x_{ij}, |1 - x_{ij}|) \cdot c_r$$

Việc áp đặt ràng buộc nhánh được thực hiện bằng cách điều chỉnh trực tiếp ma trận dist:

```

def edges_based_on_branching(user_param, branching, recur):
    if branching.branch_value == 0:    # Cam canh (i,j)
        user_param.dist[i][j] = user_param.verybig
    else:                               # Bat buoc canh (i,j)
        # Cam tat ca canh ra khoi i, tru (i,j)
        user_param.dist[i][:] = user_param.verybig
        user_param.dist[i][j] = user_param.dist_base[i][j]
        # Cam tat ca canh vao j, tru (i,j)

```

```

user_param.dist[:, j] = user_param.verybig
user_param.dist[i][j] = user_param.dist_base[i][j]
# Cam canh nguoc (j,i)
user_param.dist[j][i] = user_param.verybig
if recur:
    edges_based_on_branching(user_param, branching.
                             father, recur)

```

Nhờ cơ chế đệ quy (recur=True), khi bắt đầu xử lý một nút mới, toàn bộ chuỗi ràng buộc từ nút gốc đến nút hiện tại được tái áp dụng lên ma trận dist, đảm bảo tính nhất quán của mô hình.

6.5.2 Nguyên lý hoạt động

Thuật toán B&B trong cài đặt này sử dụng tìm kiếm theo chiều sâu (Depth-First Search), với cây B&B được biểu diễn qua cấu trúc TreeBB:

```

class TreeBB:
    father          # nut cha
    son0            # nut con dau tien
    branch_from     # nhan i cua canh phan nhanh
    branch_to       # nhan j cua canh phan nhanh
    branch_value    # 0 = cam canh, 1 = bat buoc canh
    lowest_value    # gia tri cgan duoi tai nut nay
    toplevel       # danh dau nut cap cao nhat hien tai

```

Cập nhật cận:

- **Cận dưới (Lower Bound):** Được cập nhật tại các nút cấp cao nhất (toplevel), lấy giá trị nhỏ nhất giữa hai nhánh con:

$$LB = \min(\text{son0.lowest_value}, \text{son1.lowest_value})$$

- **Cận trên (Upper Bound):** Được cập nhật khi tìm thấy nghiệm nguyên khả thi tốt hơn.

Điều kiện cắt tỉa:

- **Gap đủ nhỏ:** $\frac{UB - LB}{UB} < \varepsilon$, kết thúc toàn bộ thuật toán.
- **Vô nghiệm:** Giá trị CG $> 2 \times \text{maxlength}$ hoặc $< -\varepsilon$.
- **Bound cut:** Giá trị CG tại nút $> UB$, cắt tỉa nhánh này.

Pseudocode Branch and Price:

```

Input:  user_param (do thi, rang buoc), init_routes
Output: best_routes (cac tuyen duong toi uu), UB (chi phi
        toi uu)

BranchAndPrice():
    UB =  $+\infty$ , LB =  $-\infty$ 
    best_routes = []
    bb_node(user_param, init_routes, root=None, best_routes,
            depth=0)

bb_node(user_param, routes, branching, best_routes, depth):
    // Kiem tra gap
    if (UB - LB) / UB < gap: return True

    // Ap dung rang buoc phan nhanh len dist
    edges_based_on_branching(user_param, branching, recur=
        False)

    // Giai Column Generation tai nut nay
    cg_obj, routes = ColumnGeneration(routes).
        compute_col_gen()

    // Kiem tra tinh kha thi cua relaxation
    if cg_obj > 2*maxlength or cg_obj < 0: return True //
        Prune: infeasible

```



```
branching.lowest_value = cg_obj
cap nhat LB neu can

if cg_obj > UB: return True // Prune: bound cut

// Kiem tra nguyen va chon canh phan nhanh
tinh ma tran edges tu cac route hien tai

feasible = True
for (i,j) in cac canh:
    if 0 < edges[i][j] < 1:
        feasible = False
        chon (i*, j*) theo tieu chi tot nhat

if feasible:
    if cg_obj < UB: UB = cg_obj; luu best_routes //
        Prune: optimality
    return True

// Phan nhanh: tao hai nut con
// Nhanh 1: branch_value = best_val (0 hoac 1)
node1 = TreeBB(branching, i*, j*, best_val)
cap nhat dist theo node1 (khong de quy)
loc routes phu hop voi node1
bb_node(user_param, filtered_routes1, node1, best_routes
    , depth+1)
branching.son0 = node1

// Nhanh 2: branch_value = 1 - best_val
phuc hoi dist_base
node2 = TreeBB(branching, i*, j*, 1-best_val)
cap nhat dist theo toan bo chuoai phan nhanh (de quy)
```

```

loc routes phu hop voi node2
bb_node(user_param, filtered_routes2, node2, best_routes
        , depth+1)

branching.lowest_value = min(node1.lowest_value, node2.
        lowest_value)
return True

```

6.6 Độ phức tạp của thuật toán

- Gọi $n = |N|$ là số khách hàng, K là số xe, và $|R|$ là số cột (tuyến đường) trong RMP.
- **Column Generation tại mỗi nút:** Mỗi vòng lặp giải một LP có $|R|$ biến và n ràng buộc, chi phí $O(n \cdot |R|^{1.5})$ theo thuật toán Simplex. SPPRC có độ phức tạp giả đa thức $O(2^n \cdot n^2)$ trong trường hợp xấu nhất do không gian trạng thái visited.
- **Cây B&B:** Số nút tối đa là $O(2^{n^2})$ (số biến nhị phân x_{ij}), tuy nhiên nhờ cắt tỉa và Column Generation, số nút thực tế thường nhỏ hơn rất nhiều.
- **Độ phức tạp tổng thể (worst case):**

$$T(n) = O\left(2^{n^2} \cdot 2^n \cdot n^4\right)$$

Tuy nhiên, trong thực tế với các bộ dữ liệu chuẩn Solomon (25–100 khách hàng), thuật toán hội tụ nhanh nhờ kỹ thuật cắt tỉa, dominance trong SPPRC, và chất lượng của cận dưới từ Column Generation.

7 KẾT QUẢ THỰC NGHIỆM

Chúng tôi tiến hành so sánh hiệu suất của ba phương pháp: Branch-and-Bound (B&B), Branch-and-Cut (B&C) và Branch-and-Price (B&P) trên các instance Solomon benchmark. Tất cả các thuật toán đều được chạy với cùng bộ tham số (Vehicle = 25, Capacity = 200).

7.1 Thiết lập thực nghiệm

- Số lượng khách hàng: 5, 10, 15, 25 (bao gồm depot tại nút 0)
- Số lượng phương tiện: 25 xe

7.2 Kết quả so sánh

Bảng 1 trình bày kết quả thực nghiệm tổng hợp của ba phương pháp.

Method	$N = 5$		$N = 10$		$N = 15$		$N = 100$	
	Obj↓	Time	Obj↓	Time	Obj↓	Time	Obj↓	Time
C101								
B&B	42.4	0.01s	58.2	59.80s				
B&C	42.4	0.06s	58.2	29.68s				
B&P	42.4	0.01s	58.2	0.2s	142.1	0.44s	-	-
R101								
B&B	156.3	0.03	269.4	18.92s				
B&C	156.3	0.07	269.4	11.95s				
B&P	156.3	0.01	269.4	0.02s	383.7	0.03s	1643.0	151.84s
RC101								
B&B	89.2	0.06	185.9	20.90s	...			
B&C	89.2	0.11	185.9	19.85s				
B&P	89.2	0.01s	185.9	0.11s	228.4	0.26s	-	-

Bảng 1: Kết quả so sánh các thuật toán với quy mô khách hàng nhỏ và trung bình

7.3 Phân tích kết quả

Như kết quả trình bày trong Bảng 1, Branch-and-Cut cho thấy sự cải thiện rõ rệt so với Branch-and-Bound truyền thống. Cụ thể:

- Branch-and-Cut giảm đáng kể số nút B&B cần khám phá nhờ các mặt cắt capacity/subtour được thêm động.
- Thời gian chạy của Branch-and-Cut nhanh hơn so với Branch-and-Bound trên cùng instance.
- Branch-and-Price cho kết quả chất lượng cao trên các instance lớn hơn nhờ cơ chế sinh cột (column generation), nhưng tốn nhiều bộ nhớ và thời gian hơn trên môi trường Python + Gurobi.

Đối với các instance nhỏ (5 và 10 khách hàng), cả ba phương pháp đều tìm được nghiệm tối ưu. Khi số lượng khách hàng tăng lên (15–25), ưu thế của Branch-and-Cut và Branch-and-Price trở nên rõ nét hơn so với Branch-and-Bound thuần túy.

7.4 Nhận xét

- Branch-and-Cut là sự cân bằng tốt giữa chất lượng nghiệm và thời gian tính toán trong môi trường Java.
- Branch-and-Price phù hợp hơn khi giải các instance lớn nhờ khả năng khai thác cấu trúc route một cách hiệu quả.
- Việc kết hợp các kỹ thuật cutting plane đã giúp thắt chặt đáng kể LP relaxation, từ đó tăng hiệu quả cắt tỉa trong cây tìm kiếm.

Kết quả thực nghiệm khẳng định rằng việc áp dụng các cải tiến (cutting planes, upper bound tốt từ heuristic, pruning sớm) đã mang lại hiệu suất vượt trội so với phương pháp cơ bản.

8 TÀI LIỆU THAM KHẢO